LLM EVALUATION & BENCHMARKING PLATFORM

User Requirements Specification

(URS)

Document Version 1.0
September 2026

Prepared by
Monish D — 24BDS1033
Soaham S — 24BDS1021

Project: LLM Evaluation and Benchmarking Platform
Course: Software Engineering — Digital Assignment

Table of Contents

Table of Contents 2
1. Introduction 3
1.1 Purpose 3
1.2 Scope 3
1.3 Intended Audience 3
1.4 Definitions, Acronyms and Abbreviations 3
1.5 References 3
2. Overall User Description 4
2.1 User Characteristics 4
2.2 User Goals 4
2.3 User Environment 4
2.4 Assumptions and Dependencies 4
3. User Requirements 4
3.1 Model & Dataset Selection 4
3.2 Comparative & Industry-Positioning Requirements (Core Differentiator) 5
3.3 Reliability & Hallucination Requirements 5
3.4 Efficiency, Cost & Reporting Requirements 5
4. Non-Functional User Expectations 6
5. Acceptance Criteria Summary 6

1. Introduction

1.1 Purpose

This User Requirements Specification (URS) describes, from the perspective of the intended users, what the LLM Evaluation and Benchmarking Platform ("the Platform") must do. It captures user needs, goals, and expectations independent of implementation detail, and forms the basis from which the Software Requirements Specification (SRS) derives detailed functional and non-functional requirements. Unlike a general-purpose leaderboard, the Platform's defining user need is comparative positioning: telling a user not just "how good" a model is, but where it stands relative to established industry-standard models and which specific capability areas (e.g., coding, mathematical reasoning, creative writing, long-context reasoning, factual grounding) it is strong or weak in.

1.2 Scope

The Platform allows a user to register multiple LLMs (proprietary API-based models and, optionally, open-source/self-hosted models), run standardized and custom benchmark suites against them, and receive a multi-dimensional report that includes quality, reliability, hallucination, capability, efficiency, and cost metrics, together with a relative standing against reference/industry-baseline models and a strengths/weaknesses capability profile per model.

1.3 Intended Audience

- ML/AI engineers and application developers choosing a model for a product feature.
- Research teams benchmarking new or fine-tuned models against established baselines.
- Engineering managers making cost/quality/latency trade-off decisions.
- Students and academic evaluators (this project's primary near-term users).
- QA/reliability teams who need hallucination and factuality tracking over time.

1.4 Definitions, Acronyms and Abbreviations

Term	Definition
LLM	Large Language Model
Baseline / Reference Model	A well-known industry-standard model (e.g., GPT-4.x, Claude, Gemini) used as the comparison anchor for relative positioning
Capability Profile	A per-model breakdown of relative strength/weakness across task categories (coding, math, reasoning, creative writing, summarization, long-context, factuality, etc.)
Hallucination	Fabricated, factually incorrect, or unsupported content generated by a model
LLM-as-a-Judge	Using a separate LLM to score or compare the outputs of the models under test
Benchmark Run	A single execution of a defined dataset + prompt set + model set + evaluation configuration

1.5 References

Digital Assignment 1 report — "LLM Evaluation & Benchmarking Platform" (Monish D, Soaham S), which supplies the domain background, motivation, and preliminary evaluation-parameter taxonomy on which this URS builds.

2. Overall User Description

2.1 User Characteristics

Primary users are technically literate (able to read benchmark tables and charts) but are not necessarily ML researchers; the interface must therefore translate raw metrics into plain-language positioning statements (e.g., "This model performs on par with GPT-4-class models for coding but is 30% more prone to hallucination on open-domain factual QA").

2.2 User Goals

1. Compare several candidate LLMs on identical tasks under identical conditions.
2. Understand, in relative terms, how each candidate stands against known industry-standard models — not just an absolute score.
3. Identify the specific capability categories in which a model excels or underperforms, so the right model can be picked for the right sub-task (e.g., a cheap model for routine summarization, a stronger model reserved only for reasoning-heavy calls).
4. Track a model's hallucination and factual-reliability behavior separately from raw task accuracy.
5. Understand the cost and latency implications of a choice, not only its quality.
6. Reproduce a past benchmark run exactly, to validate whether a newer model version has improved or regressed.
7. Export a shareable, presentable report for a team or academic submission.

2.3 User Environment

The Platform is accessed through a standard web browser on a laptop/desktop. Users may have limited or no budget for paid model APIs, so the environment must work with free-tier and open-source model access (e.g., OpenRouter free models, self-hosted models via free inference credits) alongside any paid APIs the user chooses to add.

2.4 Assumptions and Dependencies

- Users have valid API keys (or use free-tier equivalents) for at least one LLM provider.
- Reference/baseline model results may need to be pre-computed and cached because continuously re-running expensive proprietary baselines is not always affordable.
- Internet connectivity is required for any hosted-model evaluation.

3. User Requirements

Each requirement is expressed as a user story with an associated priority (Must / Should / Could, MoSCoW) and acceptance criteria.

3.1 Model & Dataset Selection

ID	User Story	Priority
UR-01	As a user, I want to select one or more LLMs from a provider list so that I can benchmark them together.	Must
UR-02	As a user, I want to pick from prebuilt benchmark datasets (coding, math, reasoning, summarization, creative writing, factual QA, long-context) so I don't have to build my own.	Must
UR-03	As a user, I want to upload a custom dataset/prompt set so that I can test models on my own use case.	Should
UR-04	As a user, I want to configure temperature, max tokens, and system prompt so results reflect my intended production settings.	Should

3.2 Comparative & Industry-Positioning Requirements (Core Differentiator)

ID	User Story	Priority
UR-05	As a user, I want each evaluated model automatically compared against a curated set of industry-standard baseline models so I know how it ranks, not just its raw score.	Must
UR-06	As a user, I want a capability radar/profile per model, showing relative strength across categories such as coding, mathematics, reasoning, creative writing, summarization, factual grounding, and long/out-of-context reasoning.	Must
UR-07	As a user, I want plain-language "positioning statements" (e.g., "comparable to GPT-4-class on coding, below Claude-class on long-context reasoning") generated automatically.	Should
UR-08	As a user, I want to see the percentile or tier (e.g., Frontier / Strong / Mid / Weak) of a model per category rather than only a numeric score.	Should
UR-09	As a user, I want historical tracking so I can see if a model's relative standing improves after a provider update.	Could

3.3 Reliability & Hallucination Requirements

ID	User Story	Priority
UR-10	As a user, I want a hallucination rate and claim-level hallucination breakdown per model so I can judge trustworthiness, not just fluency.	Must
UR-11	As a user, I want to inspect the specific claims flagged as unsupported/incorrect, with evidence, so I can verify the evaluator's judgment.	Should

3.4 Efficiency, Cost & Reporting Requirements

ID	User Story	Priority
UR-12	As a user, I want latency, token usage, and estimated cost tracked per request so I can weigh quality against operating cost.	Must
UR-13	As a user, I want a consolidated benchmark report with charts and tables I can export (PDF/CSV) so I can share results.	Must
UR-14	As a user, I want to see a "cost-vs-performance" plot so I can identify the best value model for my budget.	Should

ID	User Story	Priority
UR-15	As a user, I want benchmark configuration stored so that results are reproducible.	Must
UR-16	As a user, I want a failed model call to not stop the whole benchmark run.	Must

4. Non-Functional User Expectations

- Usability: a non-expert should be able to configure and launch a benchmark within a few minutes.
- Transparency: every score must be traceable to the underlying raw response and evaluation method used.
- Affordability: the platform itself must run on free-tier infrastructure (hosting, database, inference where possible).
- Trustworthiness of comparison: baseline/reference model results must be clearly labeled with the version and date they were captured, since "industry standard" is a moving target.

5. Acceptance Criteria Summary

The URS is considered satisfied when a user can: (a) select two or more models and a benchmark dataset, (b) execute a run, (c) view per-model quality, reliability/hallucination, capability, efficiency, and cost metrics, (d) see each model positioned against at least one industry-standard baseline with a capability strengths/weaknesses breakdown, and (e) export the results as a report.